

TPM SPI Link Encryption 작업 내용 (요약)

1. 개요

항목	내용
목적	TPM SPI 버스를 지나는 비밀 데이터와 난수를 암호화해 버스 도청을 막음
브랜치	tpm-spi-encryption (기존 upstream/release-toe28d4757a3)
커밋	4건: e24c12e9, 0186b2d8, 10fddfff, 251d36f4

2. 작업 전 상태

- 사용자 공간 tpm2-tools의 암호화 세션은 이미 적용돼 있었음: rootfs/overlay/data FDE 키, OTA 재봉인, VPN PKCS#11 PIN.
- 보호되지 않던 부분:
 1. VPN PSK/PPK 봉인/해제가 평문으로 전송됨
 2. PKCS#11 토큰/키 프로비저닝(tpm2_ptool)에서 토큰 wrapping key가 평문으로 해제됨
 3. 커널의 TPM 트래픽(hwrng GetRandom)이 평문으로 전송됨 (커널 6.6.116에는 세션 기능이 없음)
 4. 일부 키·PIN 생성이 /dev/urandom 직접 읽기로 되어 있었음

3. 변경 내역

3.1 VPN PSK/PPK 봉인/해제 (e24c12e9)

- tpm-seal-secret.sh: tpm2_create를 암호화 HMAC 세션으로 감쌌(입력 비밀 암호화)
- tpm-unseal-secret.sh: 정책 세션을 salted 암호화 세션으로 교체 (해제 응답 암호화)

3.2 난수 생성 일원화 (0186b2d8)

- 남은 /dev/urandom 직접 사용 3곳을 shiba_get_random으로 교체
 - 동작: 커널 CRNG 시드를 기다린 뒤 OpenSSL FIPS CTR_DRBG로 생성
 - 대상: OTA rootfs LUKS 키(update-rootfs-luks-verity-with-tpm.sh), VPN PIN(vpn-pkcs11-pin.sh), FE salt(fe-lib.sh)

3.3 PKCS#11 프로비저닝 (10fddfff)

- 토큰과 키 생성을 tpm2_ptool addtoken/addkey에서 pkcs11-tool(libtpm2_pkcs11)로 바꿈. 라이브러리 내부에서 salted+bound HMAC 세션을 씀.
- 기존 동작은 그대로 유지
 - CKA_ID 형식, 키 속성(Usage/Access)이 ptool과 같음 → swan-set-ckaid.sh 수정 불필요
 - 지원 알고리즘: ecc256 / ecc384 / rsa3072
- TPM2_PKCS11_LOG_LEVEL=0으로 사용하지 않는 FAPI 경고 제거 (오류 메시지는 유지)

3.4 커널 TPM 세션 백포트 (251d36f4)

- 커널 6.10으로 올리는 대신 6.6.116에 TPM HMAC 세션 기능(CONFIG_TCG_TPM2_HMAC)을 백포트함 (ST BSP는 6.6만 지원)

- 이식 기준: linux-6.12.112 (LTS) + mainline 버그 수정 4건
- 산출물: 6.6/0005-tpm-hmac-sessions.patch (신규), linux-stm32mp_%.bbappend, 6.6/fragment-tpm.cfg
- 동작
 - 부팅 시 TPM null hierarchy에 primary 키를 만들고, 이 키로 salt를 보내 세션을 맺음 (ECDH P-256, AES-128-CFB, SHA-256)
 - hwrng TPM2_GetRandom: 응답 암호화 + HMAC
 - TPM2_PCR_Extend: HMAC
 - 세션은 hwrng 호출 사이에 유지하고, 사용자 공간 명령 전에 닫음
- 디바이스에서 동작 확인. hwrng 명령당 약 24 ms (세션 없을 때 약 9 ms), 유틸리티 시 영향 없음

4. 작업 후 TPM SPI Link Encryption 적용 범위

구분	대상 (스크립트/모듈)	보호	비고
커널	hwrng TPM2_GetRandom	응답 암호화 + HMAC	이번 작업 (커널 백포트)
커널	TPM2_PCR_Extend	HMAC	이번 작업. 현재 커널에서 호출하는 곳 없음 (IMA 미사용)
사용자 공간	rootfs LUKS 키 해제 (init-dmccrypt.sh, initramfs)	응답 암호화	기존
사용자 공간	overlay-data FDE·FE 키 봉인/해제 (fde-kek-lib.sh, initramfs init-fde.sh, fe-lib.sh)	입력/응답 암호화	기존
사용자 공간	OTA rootfs 새 키 봉인·재봉인 (pcr-predict-reseal.sh)	입력/응답 암호화	기존
사용자 공간	VPN PKCS#11 PIN (vpn-pkcs11-pin.sh)	입력/응답 암호화	기존
사용자 공간	VPN PSK/PPK (tpm-seal-secret.sh, tpm-unseal-secret.sh)	입력/응답 암호화	이번 작업
PKCS#11	토큰/키 프로비저닝 (pkcs11-functions.inc)	libtpm2_pkcs11 세션	이번 작업 (tpm2_ptool 대체)
PKCS#11	VPN 인증 서명 등 런타임 사용 (libtpm2_pkcs11)	libtpm2_pkcs11 세션	기존

암호화하지 않는 TPM 트래픽:

- 비밀이 실리지 않는 명령: PCR 읽기, 핸들/속성 조회, 공개키 조회, NV 카운터 등
- 부트로더(TF-A, U-Boot)의 PCR extend (5장 참조)

5. 제한 사항

- 수동 도청만 막음 (6장 참조).
- 커널 세션 암호(ECDH P-256, AES-128-CFB, HMAC-SHA-256)는 OpenSSL FIPS 경계 밖. 전송 보호용이고 키 생성에는 쓰이지 않음.

- 부트로더(TF-A, U-Boot)의 PCR extend는 적용 범위 밖.

6. 능동 인터포저 방어

구분

공격	방법	현재
수동 도청	SPI 선에 프로브를 물려 오가는 데이터를 기록만 함	막음
능동 인터포저	CPU와 TPM 사이에 칩을 끼워 넣어 명령·응답을 가로채고, 바꾸고, TPM인 척 응답함 (보드 개조 필요)	못 막음

능동 인터포저 방어란, 세션을 맺는 상대가 진짜 TPM인지 확인해서 이런 중간 칩을 무력화하는 것을 말함.

추가 작업으로 막을 수 있는 것: 엿보기·중계형 중간자 공격

- 공격: 인터포저가 가짜 TPM 키를 주고 세션 키를 알아낸 뒤 내용을 엿보거나 바꿈
- 방법: 공장에서 TPM EK 공개키 해시를 OTP 또는 OP-TEE 보안 저장소에 고정해 두고, 부팅 때마다 대조한 뒤 그 EK를 세션 키로 사용
 - 이 보드의 TPM(ST33)에는 제조사 EK 인증서가 없어서 이 방식이 필요함
- 규모: `tpm-algo-lib.sh` 수정, `initramfs` 수정, 공장 공정 추가 (약 1~2주)

막을 수 없는 것: TPM 직접 조작

- 공격: 인터포저가 TPM을 리셋한 뒤 정상 부팅 때의 PCR 값을 그대로 다시 extend하고, 직접 unseal을 요청해 봉인된 키를 꺼냄
- 별도 칩 TPM(dTPM) 구조의 한계라서, 세션 키 검증으로는 막을 수 없음
- 근본 대책은 fTPM(OP-TEE 내 TPM), 보드 물리 보호, 비밀에 사용자 입력을 결합하는 것 (red의 data FDE·FE는 이미 passphrase를 결합하고 있어 안전함)

결론: 중간자 공격은 추가 작업으로 막을 수 있지만, TPM 직접 조작이 남으므로 “능동 인터포저 방어”라고 주장할 수는 없음. 현재 “수동 도청만 막음”으로 유지함.